동적 계획법

Dynamic Programming

네 가지 코딩 문제로 배우는 동적 계획법

기말고사는 코딩테스트로 평가합니다

공개 학습 문제는 챕터별 4개입니다.

공개 학습 범위	기말고사
6개 챕터 × 4문제 = 24문제	공개된 24문제 중 10문제 출제
아주 쉬운 구현부터 단계적으로 연습	작성한 코드의 실행 결과로 평가
강의노트와 사이트 실습을 같은 문제로 통일	별도 객관식·서술형 시험 없이 코딩테스트

각 문제의 입력 범위, 함수 이름, 반환값과 경계 조건까지 공부합니다.

6장에서 공부할 문제는 네 개입니다

쉬운 구현에서 시작해 표의 조건을 하나씩 늘립니다.

번호	공개 문제	난도	배울 것
6-1	피보나치 수	아주 쉬움	표를 순서대로 채우기
6-2	인접하지 않은 수의 최대 합	쉬움	고르기·건너뛰기 비교
6-3	1로 만들기	기본	가능한 연산 중 최솟값
6-4	0/1 배낭	조금 도전	물건과 용량의 2차원 표

이 네 문제를 Python 또는 C로 직접 구현하고 테스트합니다.

약 2시간 수업 흐름

설명과 직접 구현 시간을 함께 확보합니다.

시간	내용
0–15분	표 한 칸의 뜻과 계산 순서
15–30분	6-1 피보나치와 직접 구현
30–50분	6-2 인접 선택 금지와 손 계산
50–60분	쉬는 시간
60–80분	6-3 1로 만들기
80–105분	6-4 0/1 배낭
105–120분	네 문제 중 한 문제 구현·수정·재제출

코드는 두 언어로 제공하되, 각자 익숙한 언어로 연습합니다.

DP로 할 수 있는 일

Dynamic Programming을 줄여서 DP라고 부릅니다.

값 구하기

예: n번째 피보나치 수

최댓값 찾기

예: 인접하지 않은 수의 최대 합, 배낭의 최대 가치

최솟값 찾기

예: 1로 만드는 최소 연산 횟수

먼저 표 한 칸의 뜻을 정합니다

dp[i]는 문제마다 다른 정보를 담습니다.

피보나치

dp[i] = i번째 피보나치 수

인접 선택 금지

dp[i] = 앞의 i개 원소에서 얻는 최대 합

1로 만들기

dp[i] = i를 1로 만드는 최소 연산 횟수

DP를 설계하는 네 가지 질문

문제마다 아래 순서로 정리합니다.

질문	확인할 내용
① 무엇을 저장하나?	dp의 한 칸을 한 문장으로 정의
② 어떻게 계산하나?	이미 아는 작은 답과 현재 답의 관계
③ 어디서 시작하나?	직접 알 수 있는 가장 작은 입력의 답
④ 어떤 순서로 계산하나?	참조하는 값이 먼저 준비되는 순서

마지막에는 시간과 메모리가 입력 범위 안에 드는지도 확인합니다.

분할 정복과 DP

대표 예제에서 부분 문제가 어떻게 나타나는지 비교합니다.

	합병 정렬	재귀 피보나치
나누는 문제	왼쪽 구간·오른쪽 구간 정렬	F(n−1)·F(n−2) 계산
같은 계산의 반복	나눈 구간들이 보통 겹치지 않음	같은 F(k)가 여러 번 등장
주된 처리	각 구간을 정렬한 뒤 합침	F(k)를 저장해 다시 사용

결과 저장 여부만으로 모든 알고리즘을 분류하지는 않습니다.

저장한 답을 다시 써도 되는 조건

현재 문제를 설명하는 정보가 충분해야 합니다.

같은 상태라면 같은 답

예: F(4)는 어디서 요청해도 3입니다.

작은 답으로 현재 답을 계산

최적화 문제에서는 가능한 선택을 비교해 최선의 값을 고릅니다.

같은 상태가 반복 등장

저장해 두면 중복 계산을 줄일 수 있습니다.

피보나치 수

아주 쉬움

정의

F(0)=0, F(1)=1. n≥2이면 F(n)=F(n–1)+F(n–2)입니다.

입력

정수 n, 0≤n≤90

구할 값

n번째 피보나치 수 F(n)를 반환합니다.

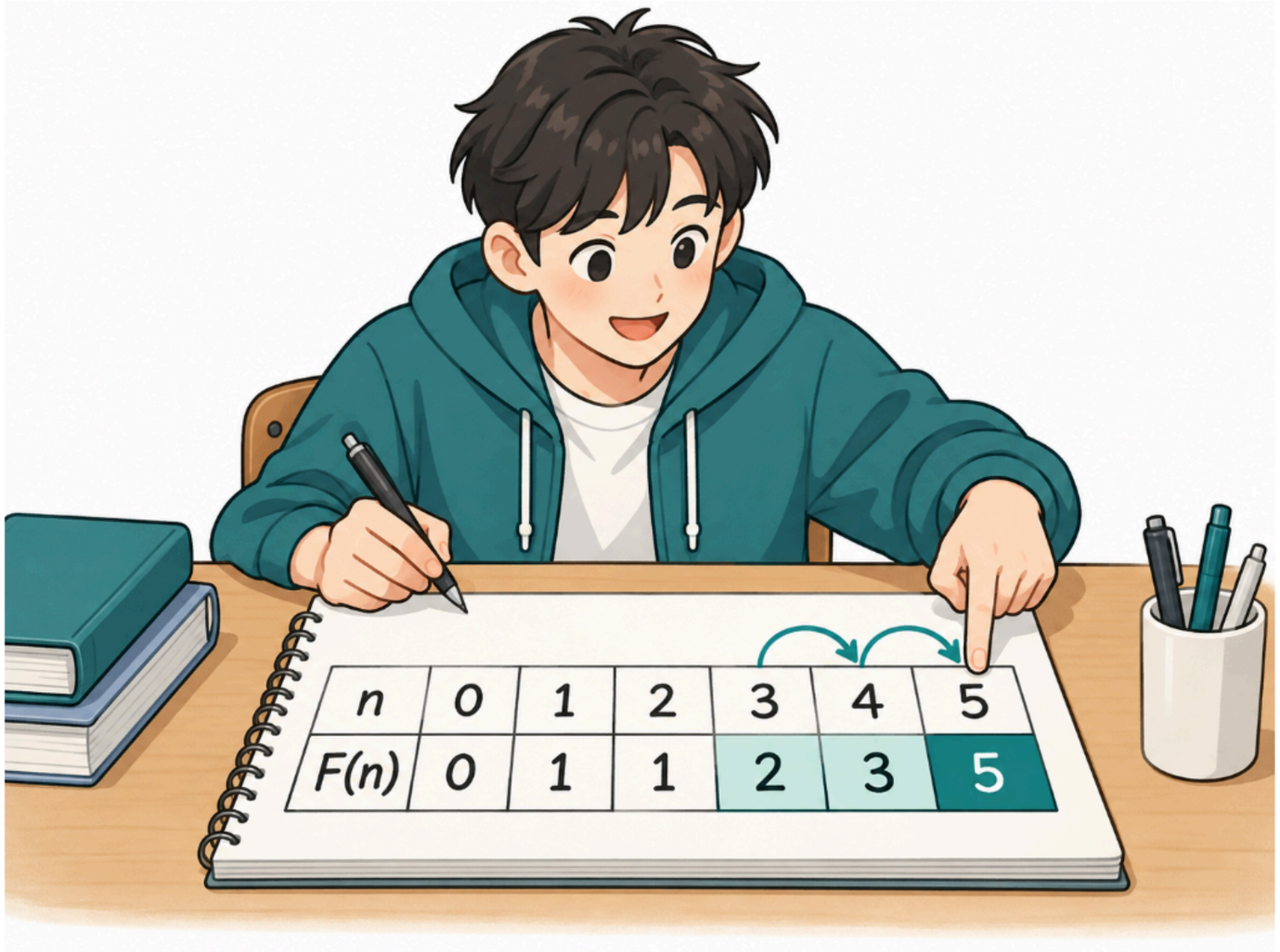
함수와 반환값

Python: fib_bottomup(n) / C: long long fib_bottomup(int n)

입력 n	반환값
0	0
1	1
7	13
10	55

함수의 반환값을 실행 결과로 평가합니다.

앞의 두 값을 더해 새 칸에 적습니다



F(5) = 3 + 2 = 5
앞의 두 값을 더해
새 칸에 적습니다.

피보나치의 정의

F(0)=0, F(1)=1, n≥2이면 앞의 두 수를 더합니다.

n	0	1	2	3	4	5	6	7
F(n)	0	1	1	2	3	5	8	13

F(5) = F(4) + F(3) = 3 + 2 = 5

정의를 그대로 재귀로 쓰면

같은 F(k)를 여러 번 계산합니다.

F(5)를 구할 때

F(4)와 F(3)을 계산합니다.

F(4)를 구할 때

F(3)과 F(2)를 다시 계산합니다.

중복되는 계산

F(3), F(2), F(1) 등이 여러 번 등장합니다.

단순 재귀 코드

실습에서는 n=5, n=10처럼 작은 입력으로 확인합니다.

```
def fib_naive(n):
    if n < 2:
        return n
    return fib_naive(n-1) + fib_naive(n-2)
```

fib_naive(10)의 반환값은 55입니다.
시간 $O(2^n)$, 호출 스택 $O(n)$
$O(2^n)$은 지수 증가를 나타내는 상한입니다.

계산한 F(k)를 메모합니다

아직 계산하지 않은 칸은 -1로 구분합니다.

k	0	1	2	3	4	5
처음 memo[k]	-1	-1	-1	-1	-1	-1
계산 후 memo[k]	0	1	1	2	3	5

값이 이미 저장되어 있으면 그 값을 바로 반환합니다.

작은 값부터 채우는 바텀업

F(0), F(1)을 적고 2부터 n까지 계산합니다.

```
def fib_bottomup(n):
    if n < 2:
        return n
    dp = [0] * (n+1)
    dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

반복문이 현재 칸을 계산할 때
앞의 두 칸은 이미 채워져 있습니다.

재귀 호출 없이 같은 점화식을 사용합니다.

바텀업 · C

n=0, n=1을 먼저 처리한 뒤 표를 채웁니다.

```
long long fib_bottomup(int n) {
    if (n < 2) return n;
    long long dp[91] = {0};
    dp[1] = 1;
    for (int i=2; i<=n; ++i)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

입력 0≤n≤90
예: fib_bottomup(10) = 55

시간 O(n), 표의 공간 O(n)

탑다운과 바텀업 비교

같은 상태와 점화식을 다른 순서로 계산합니다.

방식	계산 순서	시간	추가 공간
단순 재귀	재귀 호출을 매번 실행	$O(2^n)$	O(n) 스택
탑다운	필요한 상태를 재귀로 요청	O(n)	O(n) 메모·스택
바텀업	작은 상태부터 반복문	O(n)	O(n) 표
바텀업 압축	직전 두 값만 유지	O(n)	O(1) 변수

외워서 고르기보다 필요한 상태와 계산 순서를 보고 결정합니다.

6-1을 코드 없이 먼저 적어 봅니다

빈칸에 들어갈 초기값과 식을 정합니다.

dp[0] = ?
dp[1] = ?
for i in 2..n:
dp[i] = ?

확인: n=0, n=1, n=2, n=10

인접하지 않은 수의 최대 합

쉬움

문제

정수 배열에서 바로 이웃한 두 원소를 함께 고르지 않습니다.

입력

배열 길이 0..10,000, 각 값 $-10^6..10^6$. 아무것도 고르지 않아도 됩니다.

구할 값

고른 원소 합의 최댓값을 반환합니다. 빈 배열 또는 모두 음수이면 0입니다.

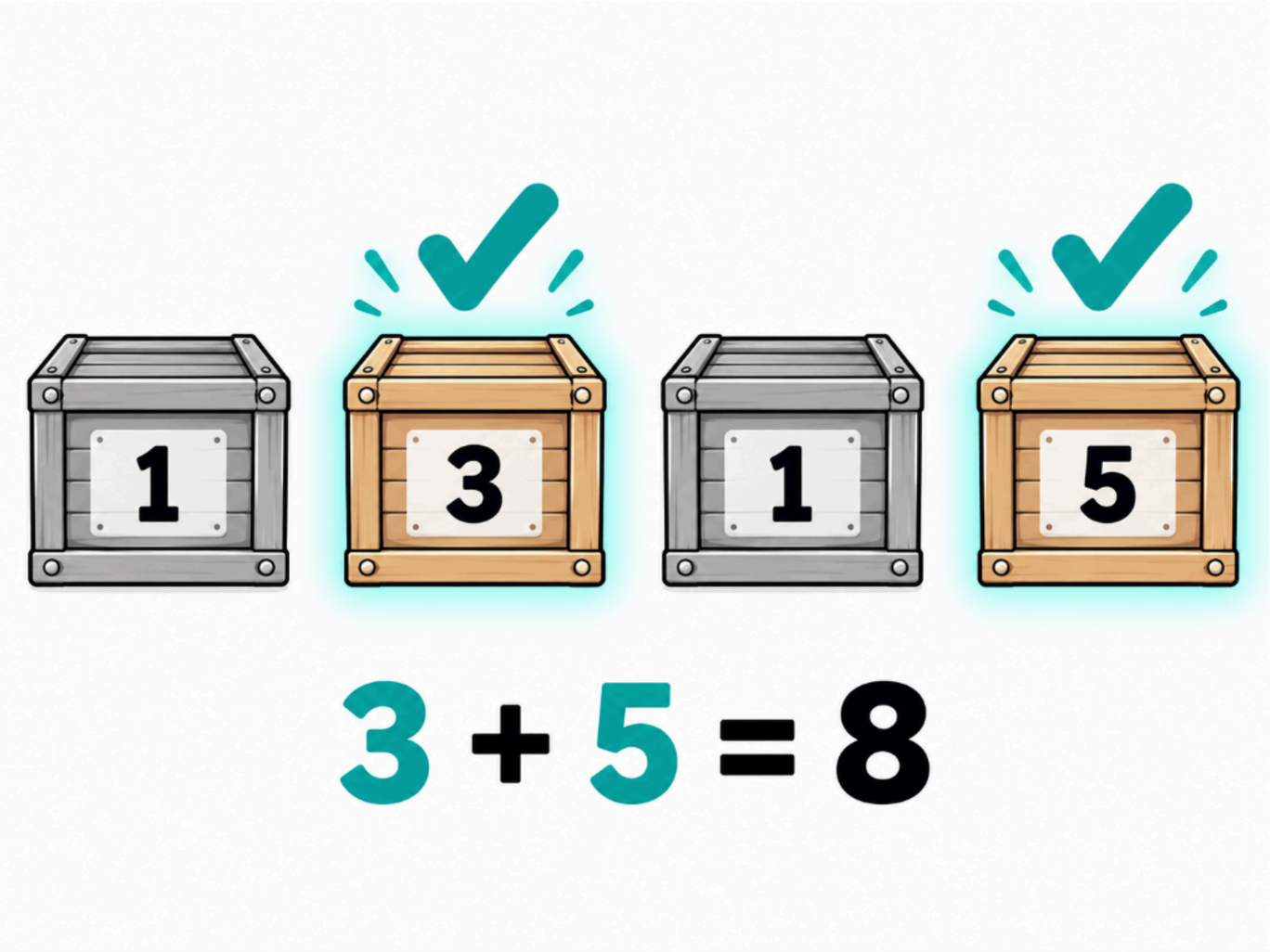
함수와 반환값

Python: nonadjacent(a) / C: long long nonadjacent(const int a[], int n)

배열 a	반환값
[1,3,1,5]	8
[5,1,1,5]	10
[-4,-2]	0
[]	0

함수의 반환값을 실행 결과로 평가합니다.

3과 5는 이웃하지 않아 함께 고를 수 있습니다



[1,3,1,5]
두 번째와 네 번째를 고릅니다.
최대 합은 3+5=8

인접한 창고를 함께 고를 수 없습니다

일렬 창고의 식량은 [1, 3, 1, 5]입니다.

조건

바로 이웃한 두 창고를 동시에 선택할 수 없습니다.

목표

선택한 식량 합의 최댓값. 예제는 3과 5를 골라 8입니다.

공개 문제 6-2의 범위

길이 0..10,000, 값은 $-10^6..10^6$. 아무것도 고르지 않아도 됩니다.

현재 창고를 고르거나 건너뜁니다

D[i] = 앞의 i개 원소에서 얻는 최대 합, D[0]=0

i	이번 값	이번 값을 안 고름	이번 값을 고름	D[i]
1	1	0	1	1
2	3	1	3	3
3	1	3	1+1=2	3
4	5	3	3+5=8	8

D[i] = max(D[i−1], D[i−2]+a[i−1]) (i≥2)
D[1] = max(0, a[0])

인접 선택 금지 최대 합 · Python

```
def nonadjacent(a):
    prev2 = prev1 = 0
    for value in a:
        current = max(prev1, prev2 + value)
        prev2, prev1 = prev1, current
    return prev1
```

예: nonadjacent([1,3,1,5])=8
예: nonadjacent([])=0, nonadjacent([-4,-2])=0
시간 O(n), 추가 공간 O(1)

인접 선택 금지 최대 합 · C

```
long long nonadjacent(const int a[], int n) {
    long long prev2=0, prev1=0;
    for (int i=0; i<n; ++i) {
        long long take=prev2+a[i];
        long long current=prev1>take ? prev1:take;
        prev2=prev1; prev1=current;
    }
    return prev1;
}
```

최대 합 예: $5{,}000 \times 10^6 = 5 \times 10^9$
중간값과 반환값을 long long으로 계산합니다.
시간 O(n), 추가 공간 O(1)

현재 원소를 고른 경우를 설명해 봅니다

a=[1,3,1,5]에서 마지막 5를 처리합니다.

5를 안 고르면: 앞의 세 원소에서의 최대 합 3
5를 고르면: 앞의 두 원소 최대 합 3에 5를 더해 8

이번 답 = max(3, 8) = 8

10분 쉬었다가 이어갑니다

다음은 5장에서 만난 0/1 배낭입니다.

앞의 두 문제는 작은 표를 순서대로 채웠습니다.

이어서 가능한 연산 중 최소 횟수를 고르고,
물건과 용량을 함께 기록하는 표를 만듭니다.

1로 만들기

기본

문제

현재 정수에서 1 빼기 또는 2·3·5로 나누기를 반복해 1을 만듭니다.

허용 조건

나눗셈은 나누어떨어질 때만 가능합니다. 연산 하나를 1회로 셉니다.

구할 값

$1 \le n \le 10{,}000$. 1이 되기까지의 최소 연산 횟수를 반환합니다.

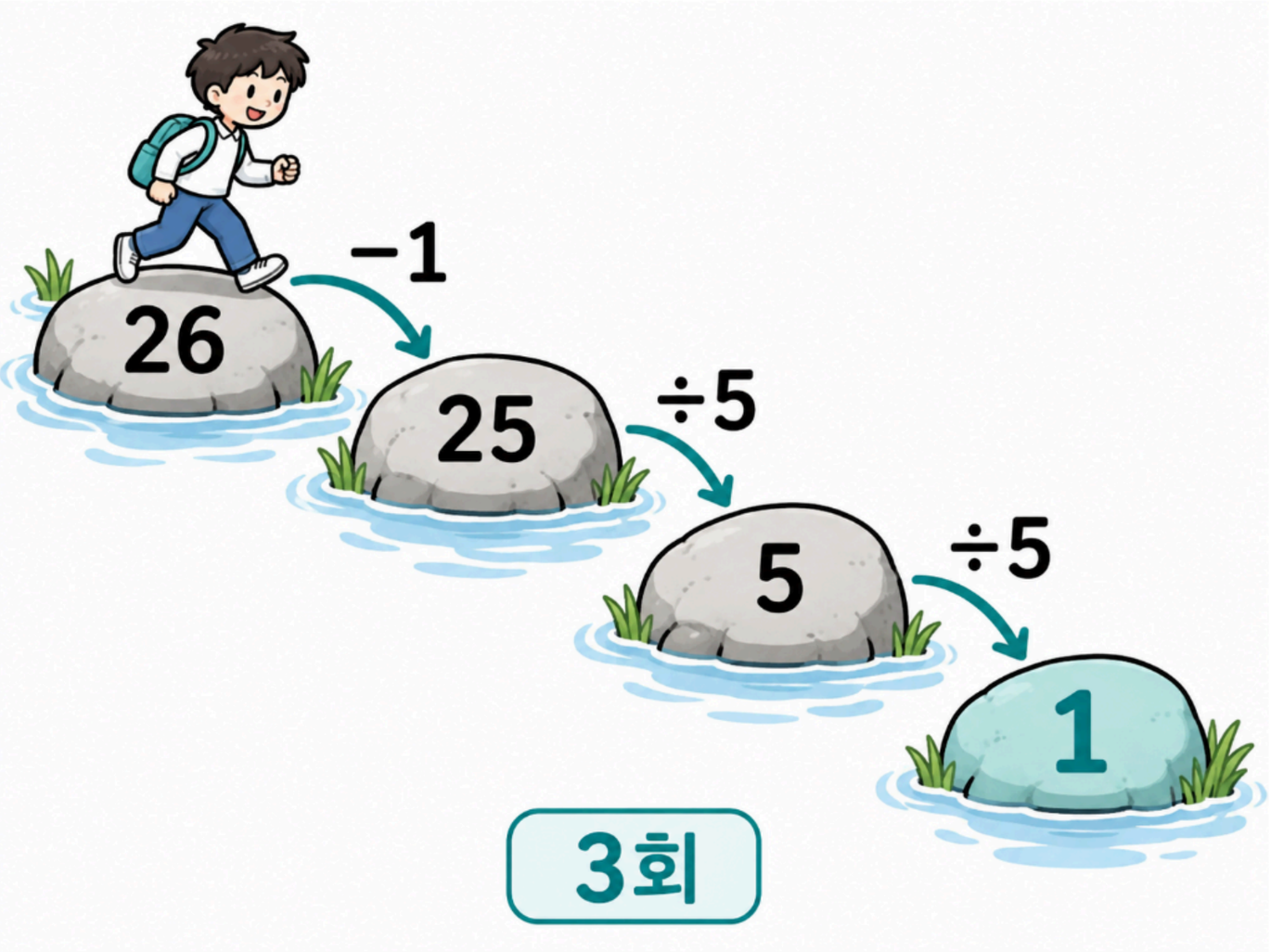
함수와 반환값

Python: make_one(n) / C: int make_one(int n, int dp[])

입력 n	반환값
1	0
10	2
13	4
26	3

함수의 반환값을 실행 결과로 평가합니다.

26에서 먼저 1을 빼면 세 번에 도착합니다



26, 25, 5, 1
세 번 이동합니다.
−1, ÷5, ÷5

정수를 1로 만드는 최소 연산 횟수

현재 값에 아래 연산 중 하나를 한 번 적용합니다.

항상 가능

1을 뺍니다.

나누어떨어질 때만 가능

2로 나누기, 3으로 나누기, 5로 나누기

구할 값

$1 \le n \le 10{,}000$. n을 1로 만드는 최소 연산 횟수

26에서 나눗셈만 먼저 고르면

전략을 '5, 3, 2 순서로 나누고, 안 되면 -1'로 정합니다.

전략	지나는 값	횟수
이 그리디 전략	26, 13, 12, 4, 2, 1	5회
더 좋은 경로	26, 25, 5, 1	3회

26에서는 1을 먼저 빼는 선택까지 비교해야 합니다.

첫 연산 뒤의 작은 문제를 봅니다

d[x] = x를 1로 만드는 최소 연산 횟수

첫 연산	가능한 조건	총 연산 횟수
1 빼기	x≥2	1 + d[x−1]
2로 나누기	x%2 = 0	1 + d[x/2]
3으로 나누기	x%3 = 0	1 + d[x/3]
5로 나누기	x%5 = 0	1 + d[x/5]

d[1]=0. 가능한 후보 중 최솟값을 고릅니다.

26의 후보를 실제로 비교합니다

작은 수부터 표를 채우면 필요한 값이 이미 있습니다.

26에서 첫 연산	남은 최소 횟수	첫 연산 포함
−1 후 25	d[25]=2	3
÷2 후 13	d[13]=4	5
÷3	사용 불가	후보에서 제외
÷5	사용 불가	후보에서 제외

d[26] = min(3, 5) = 3
1을 뺀 뒤 25, 5, 1로 가면 세 번입니다.

1로 만들기 최솟값 · Python

```
def make_one(n):
    dp = [0]*(n+1)
    for x in range(2, n+1):
        dp[x] = dp[x-1]+1
        for d in (2,3,5):
            if x % d == 0:
                dp[x] = min(dp[x], dp[x//d]+1)
    return dp[n]
```

예: make_one(1)=0, make_one(10)=2, make_one(26)=3
시간 O(n), 공간 O(n)

1로 만들기 최솟값 · C

```
int make_one(int n, int dp[]) {
    int divisors[]={2,3,5}; dp[1]=0;
    for (int x=2; x<=n; ++x) {
        dp[x]=dp[x-1]+1;
        for (int j=0; j<3; ++j) {
            int d=divisors[j];
            if (x%d==0 && dp[x/d]+1<dp[x]) dp[x]=dp[x/d]+1;
        }
    }
    return dp[n];
}
```

dp는 호출자가 길이 n+1로 준비합니다.
모든 가능한 나눗셈을 비교해야 합니다.
시간 O(n), dp를 포함한 공간 O(n)

직접 설명해 봅시다

코드를 보지 않고 네 가지 질문에 답합니다.

① make_one에서 d[7]은 무엇을 뜻하나요?
② 나누어떨어지지 않는 ÷3을 후보에 넣으면 왜 틀리나요?
③ d[1]은 왜 0인가요?
④ 2부터 차례대로 채워도 되는 이유는 무엇인가요?

0/1 배낭

조금 도전

문제

물건은 각각 하나씩만 있고 쪼갤 수 없습니다. 선택한 무게의 합은 용량 W 이하여야 합니다.

입력

물건 수 0..100, W=0..1000, 무게 1..1000, 가치 $0..10^6$

구할 값

가능한 선택 중 가치 합의 최댓값을 반환합니다. 빈 선택도 허용합니다.

함수와 반환값

Python: knapsack2(items, W) / C: int knapsack2(w, v, n, W, dp)

물건 (무게,가치), 용량	반환값
[(20,14000),(15,9000),(15,9000)], 30	18000
[(2,3)], 4	3
[], 10	0

함수의 반환값을 실행 결과로 평가합니다.

30kg 배낭에는 B와 C를 함께 담습니다



B+C를 선택합니다.
15+15=30kg
9000+9000=18000G
각각 한 번씩만
담습니다.

0/1 배낭의 조건

각 물건은 하나씩만 있습니다. 통째로 넣거나 넣지 않습니다.

물건	무게	가치	무게당 가치
A	20kg	14,000G	700G/kg
B	15kg	9,000G	600G/kg
C	15kg	9,000G	600G/kg

용량 30kg 이하에서 총 가치를 최대로 만듭니다.
남는 공간이 있어도 됩니다. 물건을 쪼개거나 반복해서 넣을 수 없습니다.

가능한 조합을 작은 예에서 비교합니다

용량 30kg을 넘는 조합은 후보가 될 수 없습니다.

선택	총 무게	총 가치	가능 여부
없음	0kg	0G	가능
A	20kg	14,000G	가능
B 또는 C	15kg	9,000G	가능
B + C	30kg	18,000G	가능
A + B 또는 A + C	35kg	23,000G	불가
A + B + C	50kg	32,000G	불가

가성비 그리디: A의 14,000G. 최적: B+C의 18,000G.

한 칸에는 두 가지 조건이 들어갑니다

dp[i][c] = 앞의 i개 물건으로, 용량 c 이하에서 얻는 최대 가치

행 i

어디까지의 물건을 사용할 수 있는가

열 c

담을 수 있는 총 무게의 한도

목표 칸

물건 수가 n, 전체 용량이 W이면 dp[n][W]

이번 물건을 안 넣거나, 넣거나

i번째 물건의 무게는 w, 가치는 v라고 하겠습니다.

선택	참조할 작은 문제	후보 가치
안 넣음	앞의 i−1개, 용량 c	dp[i−1][c]
넣음 (w≤c일 때)	앞의 i−1개, 용량 c−w	dp[i−1][c−w] + v

두 후보 중 큰 값이 dp[i][c]입니다.
넣는 경우도 이전 행을 참조하므로 같은 물건을 두 번 사용하지 않습니다.

작은 용량 표를 채웁니다

5kg의 배수 열만 발췌했습니다. 각 값은 G 단위입니다.

고려한 물건 / c	0	5	10	15	20	25	30
없음	0	0	0	0	0	0	0
A	0	0	0	0	14000	14000	14000
A,B	0	0	0	9000	14000	14000	14000
A,B,C	0	0	0	9000	14000	14000	**18000**

물건이 없으면 0, 용량이 0이어도 0입니다. 무게는 모두 양수입니다.

마지막 칸 18,000은 어디서 왔나요?

C는 무게 15kg, 가치 9,000G입니다.

dp[3][30]의 선택	계산	값
C를 안 넣음	dp[2][30]	14,000
C를 넣음	dp[2][15] + 9,000	18,000

dp[3][30] = max(14,000, 18,000) = 18,000
C를 넣고, 앞의 물건 중에서는 B를 넣은 결과입니다.

2차원 0/1 배낭 · Python

```
def knapsack2(items, W):
    dp = [[0]*(W+1) for _ in range(len(items)+1)]
    for i, (w,v) in enumerate(items, 1):
        for c in range(W+1):
            dp[i][c] = dp[i-1][c]
            if w <= c:
                dp[i][c] = max(dp[i][c], dp[i-1][c-w]+v)
    return dp[len(items)][W]
```

입력 items=[(20,14000),(15,9000),(15,9000)], W=30
반환값 18000
시간 O(nW), 표 공간 O(nW)

2차원 0/1 배낭 · C

```
int knapsack2(const int w[], const int v[], int n, int W,
              int dp[][1001]) {
    for (int c=0; c<=W; ++c) dp[0][c]=0;
    for (int i=1; i<=n; ++i) {
        for (int c=0; c<=W; ++c) {
            dp[i][c]=dp[i-1][c];
            if (w[i-1]<=c) {
                int take=dp[i-1][c-w[i-1]]+v[i-1];
                if (take>dp[i][c]) dp[i][c]=take;
            }
        }
    }
    return dp[n][W];
}
```

n≤100, W≤1000, 무게는 양수, 각 가치 0..10^6
dp[101][1001]은 호출자가 제공합니다.
반환값 최대 10^8, 시간 O(nW), 표 공간 O(nW)

O(nW)의 의미

표의 칸 수와 한 칸을 채우는 비용을 곱합니다.

칸의 수

$(n+1)\times(W+1)$

한 칸의 계산

안 넣기와 넣기 두 후보 비교, O(1)

전체

시간 O(nW), 표 공간 O(nW)

6장의 네 문제와 핵심 계산

같은 네 문제를 강의노트와 사이트에서 연습합니다.

문제	저장할 값	다음 칸 계산
6-1 피보나치	n번째 수열 값	앞의 두 값의 합
6-2 인접 선택 금지	앞부분의 최대 합	안 고름·고름의 max
6-3 1로 만들기	각 수의 최소 연산 횟수	허용 연산 후보의 min
6-4 0/1 배낭	물건·용량별 최대 가치	안 넣음·넣음의 max

문제마다 상태 정의, 초기값, 계산 순서를 먼저 확인합니다.

네 문제 중 하나를 끝까지 구현합니다

입력과 예상 반환값을 적고 테스트합니다.

첫 단계

6-1 또는 6-2를 빈 코드에서 작성합니다.

다음 단계

6-3 또는 6-4의 표를 손으로 채운 뒤 구현합니다.

마지막 확인

정상 예제, 가장 작은 입력, 조건이 갈리는 입력으로 다시 실행합니다.

공개 24문제의 함수부터 익숙하게 만듭니다

기말은 공개 문제 중 10문제의 코드 실행 결과로 평가합니다.

문제 조건과 입력 범위를 읽습니다.
함수 이름·인자·반환값을 지킵니다.
직접 구현한 뒤 여러 입력으로 실행합니다.
틀린 입력을 고치고 빈 코드에서 다시 작성합니다.

작은 문제의 답을 어떻게 다시 쓰나요?

작은 답을 저장하고, 다음 계산에서 다시 사용합니다.

6-1 피보나치
6-2 인접하지 않은 수의 최대 합
6-3 1로 만들기
6-4 0/1 배낭